计算机系统 I：组合逻辑设计 (Combinational Logic Design)
复习讲义

目录

1 Verilog HDL 简介 (Introduction to Verilog HDL) 2
1.1 背景与 HDL 概念 (Background & What is HDL) . . . 2
1.2 基于 HDL 的设计流程 (HDL-based Design Flow) . . . 2
1.3 Verilog 词法规则 (Lexical Conventions) . . . 3
1.4 基本语法 (Basic Syntax) . . . 4
1.5 建模方法 (Modeling Methods) . . . 6

2 组合逻辑电路基础 (About Combinational Logic Circuits) 8
2.1 数字逻辑电路 (Digital Logic Circuit) . . . 8
2.2 组合逻辑电路定义 . . . 8
2.3 组合逻辑与时序逻辑的区别 . . . 8
2.4 组合逻辑电路的结构约束 . . . 8
2.5 组合电路的设计选择 . . . 9
2.6 组合逻辑设计流程 (Design Procedure) . . . 9

3 组合逻辑设计示例 (Design Examples) 9
3.1 示例 1：三开关控制一盏灯 . . . 9
3.2 示例 2：BCD 到余 3 码转换器 . . . 10

4 经典组合逻辑功能块 (Classic/Basic Designs) 12
4.1 功能与功能块 (Functions and Functional Blocks) . . . 12
4.2 基本逻辑功能 (Rudimentary Logic Functions) . . . 13
4.3 使能功能 (Enabling Function) . . . 13
4.4 译码器 (Decoder) . . . 13
4.5 用译码器和 OR 门实现组合逻辑 . . . 15
4.6 七段显示译码器 (Seven-Segment Display Decoder) . . . 16
4.7 编码器 (Encoder) . . . 17

4.8 多路选择器 (Multiplexer) 18
4.9 用 MUX 实现组合逻辑 19
4.10 半加器与全加器补充 (Half Adder & Full Adder) 20

5 时序分析 (Timing Analysis) 20
5.1 电路延迟 (Circuit Delay) 20
5.2 传播延迟与污染延迟 20
5.3 关键路径与最短路径 20
5.4 竞争冒险与毛刺 (Race Hazard & Glitch) 21

6 逻辑电路分析 (Analysis of Logic Circuits) 21
6.1 分析步骤 21
6.2 示例：分析一个组合逻辑电路 21

7 复习小结 (Summary) 22

1 Verilog HDL 简介 (Introduction to Verilog HDL)

1.1 背景与 HDL 概念 (Background & What is HDL)

- **异构计算 (Heterogeneous Computing)**：现代系统常同时使用 CPU、GPU、FPGA、ASIC 等不同计算单元。CPU 灵活，GPU 擅长大规模并行，FPGA 可重构，ASIC 面向固定功能且性能/功耗最优。
- **数字电路设计**：本课程重点从逻辑功能出发，理解组合逻辑、时序逻辑以及它们如何被映射到 FPGA 或 ASIC。
- **硬件描述语言 (Hardware Description Language, HDL)**：用形式化方法描述数字电路和数字逻辑系统的语言。HDL 描述的是硬件结构与行为，不只是软件执行流程。
- **FPGA vs. ASIC**：FPGA 通过下载 bitstream 实现逻辑，开发周期短、可重配置；ASIC 最终 tape-out 生成芯片版图，前期成本高，但量产后面积、功耗和速度更优。

1.2 基于 HDL 的设计流程 (HDL-based Design Flow)

典型流程如下：

1. **逻辑设计 (Logic Design with HDL)**：定义模块、端口、数据类型和逻辑功能。
2. **仿真 (Simulation)**：编写 testbench，施加输入激励，观察输出是否符合预期。
3. **综合 (Synthesis)**：解析 HDL，进行多级逻辑综合，并映射到目标工艺库或 FPGA 资源。不可综合代码只能用于仿真，不能变成真实硬件。
4. **物理设计 (Physical Design)**：包括布局 (Placement)、布线 (Routing)、静态时序分析 (Static Timing Analysis)、LVS、DRC 等。
5. **最终步骤 (Final Step)**：ASIC 进入 tape-out；FPGA 下载 bitstream 文件到器件。

最小模块示例

```
module top(
    input  a,
    input  b,
    output c
);
assign c = a & b;
endmodule
```

其中 `module` 开始模块定义，端口列表给出输入输出，`assign` 是连续赋值语句，`endmodule` 结束模块。

Testbench 示例

```
module sim_top();
reg  a_in, b_in;
wire c_out;

top dut(
    .a(a_in),
    .b(b_in),
    .c(c_out)
);

initial begin
    a_in = 1'b0;
    b_in = 1'b0;
    #2 b_in = 1'b1;
    #2 a_in = 1'b1;
    #2 $finish;
end

always @(*) begin
    $display("a=%d,b=%d,c=%d", a_in, b_in, c_out);
end
endmodule
```

复习提示：*testbench* 中的延时、显示和激励通常用于仿真，不代表一定可综合。

1.3 Verilog 词法规则 (Lexical Conventions)

- Verilog 与 C 语言相似，**大小写敏感**，所有关键字均为小写。
- Verilog 程序由 token 组成，包括空白符、注释、分隔符、数字、字符串、标识符和关键字。
- **空白符**包括空格、制表符和换行。除字符串内部和分隔 token 外，空白一般被忽略。
- **注释**与 C 相同：`//` 为单行注释，`/* ... */` 为多行注释。嵌套注释是否可用取决于编译器，通常不推荐。

数字表示 (Number Specification)

Verilog 数字常量

- **有位宽数字 (Sized Numbers)**：格式为 `<size>'<base><number>`。例如 `4'b1111`、`12'habc`、`16'd255`。
- **基数 (Base)**：`d/D` 表示十进制，`b/B` 表示二进制，`o/O` 表示八进制，`h/H` 表示十六进制。
- **无位宽数字 (Unsized Numbers)**：默认十进制，默认位宽至少 32 位，具体由编译器决定。例如 `23232`、`'habc`、`'o234`。
- **x 与 z**：`x` 表示未知值 (Unknown)，`z` 表示高阻态 (High-impedance)。在十六进制中一个 `x/z` 表示 4 位，八进制表示 3 位，二进制表示 1 位。
- **位宽扩展**：若指定大小大于给出的数字，最高有效位为 `x` 则用 `x` 扩展，为 `z` 则用 `z` 扩展，否则零扩展。
- **负数**：负号放在位宽前，如 `-6'd3`；`4'd-2` 非法。负值内部通常以补码存储。
- **可读性符号**：下划线可插入数字中提高可读性，如 `12'b1111_0000_1010`，但不能作为第一个字符。数字中的 `?` 与 `z` 等价。

字符串、标识符与转义标识符

- **字符串 (Strings)**：用双引号包围，如 `"Hello world!"`、`"text\tcolumn1\n"`。
- **普通标识符 (Identifiers)**：可包含字母、数字、下划线和 `$`，应以字母或下划线开头；只有系统任务能以 `$` 开头。
- **关键字 (Keywords)**：Verilog 保留标识符，如 `reg`、`input`、`module`。
- **转义标识符 (Escaped Identifiers)**：以反斜杠开头，以空白符结束，中间可以包含任意可打印字符。例如 `\a+b-c` 的实际标识符是 `a+b-c`。

1.4 基本语法 (Basic Syntax)

模块、端口和实例化

```
module model_name(port_list);
    // optional port definition
    // data type definition
    // logic function description
endmodule
```

```
model_name instance_identifier(port_related_list);
```

实例化时常用命名端口连接，如 `.a(a_in)`，有利于避免端口顺序错误。

数据类型 (Data Types)

- **Net**：表示物理连接，典型类型是 `wire`。
- **Register**：表示过程赋值目标或存储类变量，典型类型是 `reg`。在组合逻辑 `always` 块中也常用 `reg` 作为过程赋值变量。
- **向量 (Vector)**：多位信号，如 `wire [7:0] a;`、`reg [31:0] rdata, wdata;`。
- **数组 (Array)**：如 `wire b[2:0]` 或 `reg [15:0] mem[1023:0]`。
- **参数 (Parameter)**：模块局部常量，常用于位宽和固定配置。

端口类型规则

- **input**：模块定义中默认为 `wire`；实例化连接端可以是 `wire` 或 `reg`。
- **output**：模块定义中可以是 `wire` 或 `reg`；实例化连接端必须是 `wire`。
- **错误示例**：`input reg in1` 不符合普通输入端口定义习惯；数组形式端口需注意维度声明位置。

运算符与优先级

课件列出的常用运算符包括：

- 一元运算符：`+`、`-`。
- 按位运算符：`~`、`&`、`|`、`^`、`^~`。
- 算术运算符：`+`、`-`、`*`、`/`、`%`。
- 归约运算符：`&`、`~&`、`|`、`| ~`、`^`、`^~`。
- 逻辑运算符：`&&`、`||`、`!`。
- 相等和关系运算符：`==`、`!=`、`<`、`<=`、`>`、`>=`。
- 拼接、移位和条件运算符：`{ }`、`>>`、`<<`、`?:`。

1.5 建模方法 (Modeling Methods)

- **结构化建模 (Structured Modeling)**：从模块级、门级或开关级描述硬件连接。
- **数据流建模 (Dataflow Modeling)**：适合组合逻辑，使用连续赋值语句 `assign`。
- **行为建模 (Behavioral Modeling)**：核心是 `always` 块，形式为 `always @(event signal list) procedure statement`。

两位大于比较器示例

设输入为 $A = A_1A_0$、$B = B_1B_0$，输出 `A_greater_than_B`。门级图对应的逻辑为：

$$A > B = A_1\overline{B_1} + A_1A_0\overline{B_0} + A_0\overline{B_1B_0}$$

```
// Structural model
module comparator_greater_than_structural(A, B, A_greater_than_B);
    input  [1:0] A, B;
    output A_greater_than_B;
    wire B0_n, B1_n, and0_out, and1_out, and2_out;

    not inv0(B0_n, B[0]), inv1(B1_n, B[1]);
    and and0(and0_out, A[1], B1_n),
        and1(and1_out, A[1], A[0], B0_n),
        and2(and2_out, A[0], B1_n, B0_n);
    or  or0(A_greater_than_B, and0_out, and1_out, and2_out);
endmodule

// Dataflow model
module comparator_greater_than_dataflow(A, B, A_greater_than_B);
    input  [1:0] A, B;
    output A_greater_than_B;
    wire B1_n, B0_n, and0_out, and1_out, and2_out;

    assign B1_n = ~B[1];
    assign B0_n = ~B[0];
    assign and0_out = A[1] & B1_n;
    assign and1_out = A[1] & A[0] & B0_n;
    assign and2_out = A[0] & B1_n & B0_n;
    assign A_greater_than_B = and0_out | and1_out | and2_out;
endmodule
```

```
// Conditional dataflow model
module comparator_greater_than_conditional2(A, B, A_greater_than_B);
    input  [1:0] A, B;
    output A_greater_than_B;
    assign A_greater_than_B = (A > B) ? 1'b1 : 1'b0;
endmodule

// Conditional model using combinations
module comparator_greater_than_conditional(A, B, A_greater_than_B);
    input  [1:0] A, B;
    output A_greater_than_B;
    assign A_greater_than_B =
        (A == 2'b00) ? 1'b0 :
        (A == 2'b01) ? ~(B[1] | B[0]) :
        (A == 2'b10) ? ~B[1] :
        (A == 2'b11) ? ~(B[1] & B[0]) :
        1'bx;
endmodule

// Behavioral-style concise model
module comparator_greater_than_behavioral(A, B, A_greater_than_B);
    input  [1:0] A, B;
    output A_greater_than_B;
    assign A_greater_than_B = A > B;
endmodule

// Testbench for structural model
module comparator_testbench_verilog();
reg  [1:0] A, B;
wire struct_out;

comparator_greater_than_structural U1(A, B, struct_out);

initial begin
    A = 2'b10; B = 2'b00;
    #10 B = 2'b01;
    #10 B = 2'b10;
    #10 B = 2'b11;
end
endmodule
```

2 组合逻辑电路基础 (About Combinational Logic Circuits)

2.1 数字逻辑电路 (Digital Logic Circuit)

- 数字逻辑电路可以看作一个黑盒：输入是 0/1，输出也是 0/1。
- 它由互相连接的数字部件组成，包括电路元件和节点。节点可分为输入节点、内部节点和输出节点。
- 数字逻辑电路主要分为两类：组合逻辑电路 (Combinational Logic Circuit) 和时序逻辑电路 (Sequential Logic Circuit)。

2.2 组合逻辑电路定义

组合逻辑电路具有：

- m 个布尔输入；
- n 个布尔输出；
- n 个开关函数，每个函数把 2^m 种输入组合映射到对应输出。

组合逻辑的核心特征

当前输出只依赖当前输入值，不依赖过去输入，也不保存状态。因此组合逻辑是 **memory-less** 的。

2.3 组合逻辑与时序逻辑的区别

- **组合电路**：输出只取决于当前输入。
- **时序电路**：由组合逻辑和存储元件组成，输出取决于当前输入以及存储元件中保存的历史状态。
- 时序电路可理解为：组合逻辑根据当前输入和当前状态生成输出与下一状态，下一状态写入存储元件。

2.4 组合逻辑电路的结构约束

- 每个元件本身必须是组合逻辑电路。
- 一个节点只能作为一个元件的输出，避免多个驱动源冲突。
- 不允许出现反馈环路 (Loop)，否则可能形成状态或不稳定振荡。

2.5 组合电路的设计选择

- **两级逻辑 (2-level)**：通常形如与或式或或与式，结构直接，便于从最小项实现；但可能需要较大的扇入。
- **多级逻辑 (multi-level)**：通过公共因子提取降低门输入成本和面积，但可能增加路径层数。
- **权衡目标**：门延迟、扇入/扇出、面积成本和速度之间需要折中。
- **功能块 (Functional Blocks)**：常用组合功能可封装为译码器、编码器、多路选择器、加法器等模块。

2.6 组合逻辑设计流程 (Design Procedure)

1. **Specification**：若没有现成规格，先用文字或 HDL 写清输入、输出和功能。
2. **Formulation**：由规格导出真值表或初始布尔方程。
3. **Optimization**：进行两级或多级优化，画出逻辑图或给出 AND/OR/NOT 网表。
4. **Technology Mapping**：把逻辑图或网表映射到选定实现技术，如 NAND、NOR、AOI 门或 FPGA LUT。
5. **Verification**：验证最终设计是否满足规格。

3 组合逻辑设计示例 (Design Examples)

3.1 示例 1：三开关控制一盏灯

规格与真值表

房间中只有一盏灯，由三个开关 S_1, S_2, S_3 控制，每个开关都能独立改变灯的状态。设开关闭合为 1，打开为 0；灯亮为 1，灯灭为 0。

S_3	S_2	S_1	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

由真值表得到标准与或式：

$$F = \overline{S_3 S_2} S_1 + \overline{S_3} S_2 \overline{S_1} + S_3 \overline{S_2 S_1} + S_3 S_2 S_1$$

K 图中四个 1 彼此不能合并，因此两级与或形式已经无法进一步通过 K 图合并。观察函数可知它是三输入奇偶校验：

$$F = S_1 \oplus S_2 \oplus S_3$$

若工艺库中有 XOR 门，使用两个 XOR 门通常比标准与或式门数更少。

Verilog 描述

```
module lamp_control(s1, s2, s3, F);
    input  s1, s2, s3;
    output F;
    wire s1, s2, s3, f;

    assign F = (~s3 & ~s2 & s1) |
               (~s3 &  s2 & ~s1) |
               ( s3 & ~s2 & ~s1) |
               ( s3 &  s2 & s1);
endmodule
```

3.2 示例 2：BCD 到余 3 码转换器

规格

- 输入为 BCD 码 A, B, C, D，表示十进制数字 0 到 9，即 0000 到 1001。
- 输出为余 3 码 W, X, Y, Z，等于对应 BCD 数值加上二进制 0011。

- BCD 输入 1010 到 1111 不是合法十进制数字，可作为无关项 (Don't Cares)。
- 实现目标为多级电路，并映射到 NAND 等门库。

Input BCD $ABCD$	Output Excess-3 $WXYZ$
0000	0011
0001	0100
0010	0101
0011	0110
0100	0111
0101	1000
0110	1001
0111	1010
1000	1011
1001	1100

两级 K 图优化结果

$$W = A + BC + BD$$

$$X = \overline{B}C + \overline{B}D + B\overline{CD}$$

$$Y = CD + \overline{CD}$$

$$Z = \overline{D}$$

课件中对应的门输入成本为：

$$G = 7 + 10 + 6 + 0 = 23$$

多级提取优化

提取公共因子：

$$T_1 = C + D$$

得到：

$$W = A + BT_1$$

$$X = \overline{B}T_1 + B\overline{CD}$$

$$Y = CD + \overline{CD}$$

$$Z = \overline{D}$$

对应成本降低为：

$$G = 2 + 4 + 7 + 6 + 0 = 19$$

继续利用布尔变换：

$$\overline{T_1} = \overline{C + D} = \overline{C}\,\overline{D}$$

进一步写成：

$$W = A + BT_1$$

$$X = \overline{B}T_1 + B\overline{T_1}$$

$$Y = CD + \overline{T_1}$$

$$Z = \overline{D}$$

此时课件给出的成本为：

$$G = 2 + 1 + 4 + 6 + 4 + 0 = 17$$

技术映射与 Verilog

技术映射可使用包含反相器、2 输入 NAND、3 输入 NAND、2 输入 NOR、2-2 AOI 的门库。映射时核心思想是把优化后的多级表达式转换为目标门库可直接实现的结构。

```
module BCD_Excess_3(A, B, C, D, W, X, Y, Z);
    input  A, B, C, D;
    output W, X, Y, Z;
    wire A, B, C, D, W, X, Y, Z, T1;

    assign T1 = C | D;
    assign W  = A | B & C | B & D;
    assign X  = ~B & T1 | B & ~T1;
    assign Y  = C & D | ~T1;
    assign Z  = ~D;
endmodule
```

4 经典组合逻辑功能块 (Classic/Basic Designs)

4.1 功能与功能块 (Functions and Functional Blocks)

- 本节讨论的是设计中反复出现、很有用的组合逻辑功能。
- 每个常用功能通常有对应的组合电路实现，称为功能块 (Functional Block)。
- 过去这些功能块常做成 SSI、MSI、LSI 芯片；今天它们更多作为 VLSI 内部模块或 HDL 中的可综合子模块使用。

4.2 基本逻辑功能 (Rudimentary Logic Functions)

四类基本组合逻辑功能如下：

- **Value-Fixing**：固定输出 $F = 0$ 或 $F = 1$，不需要布尔运算符。
- **Transferring**：传输输入，$F = X$。
- **Inverting**：反相，$F = \overline{X}$，需要一个反相器。
- **Enabling**：使能传输，常见形式为 $F = X \cdot EN$ 或 $F = X + EN$。

X	$F = 0$	$F = X$	$F = \overline{X}$	$F = 1$
0	0	0	1	1
1	0	1	0	1

多位基本功能与总线

- 宽线表示总线 (Bus)，即一个向量信号。
- 若 $F = (F_3, F_2, F_1, F_0)$，则既可以拆成单独位，也可以取连续位段，如 $F(2:1)$，还可以取不连续位集合，如 $F(3), F(1:0)$。
- 多位传输、反相、固定值和使能可看作对每一位并行执行同一种基本功能。

4.3 使能功能 (Enabling Function)

- **Enabling** 允许输入信号通过到输出。
- **Disabling** 阻止输入通过，并把输出替换为固定值。
- 禁用时输出可为高阻态 Hi-Z、0 或 1。三态缓冲器和传输门常用于产生 Hi-Z。

4.4 译码器 (Decoder)

定义

译码 (Decoding) 是把 n 位输入码转换为 m 位输出码的过程，通常满足：

$$n \leq m \leq 2^n$$

每个合法输入码产生唯一输出码。执行译码的电路称为译码器。常见的 n-to-m line decoder 会产生 n 个输入变量的 2^n 个或更少最小项。

1-to-2 与 2-to-4 译码器

1-to-2 译码器：

$$D_0 = \overline{A}, \qquad D_1 = A$$

2-to-4 译码器：

$$D_0 = \overline{A_1 A_0}, \quad D_1 = \overline{A_1} A_0, \quad D_2 = A_1 \overline{A_0}, \quad D_3 = A_1 A_0$$

课件指出，2-to-4 译码器可由两个 1-to-2 译码器和四个 2 输入 AND 门组成。

3-to-8 译码器

A_2	A_1	A_0	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

3-to-8 译码器可由三个 1-to-2 译码器和八个 3 输入 AND 门组成。

译码器扩展 (Decoder Expansion)

1. 令 $k = n$。
2. 使用 2^k 个输出 AND 门，它们由两个较小译码器驱动。若 k 为偶数，两个译码器输出数均为 $2^{k/2}$；若 k 为奇数，一个输出 $2^{(k+1)/2}$，另一个输出 $2^{(k-1)/2}$。
3. 对每个得到的译码器重复上述过程，直到到达 1-to-2 译码器。

3-to-8 的例子中，最接近均分的拆法是一个 2-to-4 译码器和一个 1-to-2 译码器；2-to-4 又可拆为两个 1-to-2。

带使能译码器与解复用器

带使能信号 EN 的 2-to-4 译码器真值表如下：

EN	A_1	A_0	D_0	D_1	D_2	D_3
0	X	X	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1

这里的 X 表示 0 或 1。若把 EN 的值分配到四个输出之一，该电路也可视为 **解复用器 (Demultiplexer)**。

4.5 用译码器和 OR 门实现组合逻辑

若要实现 m 个 n 变量函数，可以使用：

- 各输出函数的最小项和表达式；
- 一个 n-to-2^n 译码器；
- m 个 OR 门，每个输出函数一个。

两种设计方法：

1. 根据真值表，在输出为 1 的行，把对应译码器输出连接到该函数的 OR 门。
2. 直接写出每个输出函数的最小项和，然后把这些最小项 OR 起来。

二进制一位全加器示例

输入 X, Y, Z，输出和位 S 与进位 C：

$$S(X,Y,Z) = \sum m(1,2,4,7)$$

$$C(X,Y,Z) = \sum m(3,5,6,7)$$

X	Y	Z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

用 3-to-8 译码器产生最小项 0 到 7，再把 $1, 2, 4, 7$ 接到 S 的 OR 门，把 $3, 5, 6, 7$ 接到 C 的 OR 门。

4.6 七段显示译码器 (Seven-Segment Display Decoder)

- 七段数码管的段通常命名为 a, b, c, d, e, f, g，可显示 0 到 9。
- **共阳极 (Common Anode Display, CAD)**：所有 LED 阳极接逻辑 1，各段阴极接低电平点亮，因此 $a \sim g = 0$ 表示亮，1 表示灭，即低有效 (Active Low)。
- **共阴极 (Common Cathode Display, CCD)**：所有 LED 阴极接地，各段阳极接高电平点亮，因此 $a \sim g = 1$ 表示亮，0 表示灭，即高有效 (Active High)。

以下表格按高有效段输出给出 BCD-to-seven-segment decoder：

A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1
其他输入				0	0	0	0	0	0	0

4.7 编码器 (Encoder)

8-to-3 编码器

8-to-3 编码器把 one-hot 输入 I_0 到 I_7 编成 3 位输出 $O_0O_1O_2$：

$$O_0 = I_4 + I_5 + I_6 + I_7$$

$$O_1 = I_2 + I_3 + I_6 + I_7$$

$$O_2 = I_1 + I_3 + I_5 + I_7$$

I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7	O_0	O_1	O_2
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

普通编码器有两个问题：

1. 若多个输入同时为 1，例如 I_3 和 I_6 同时为 1，输出不再唯一。
2. 全部输入为 0 与 $I_0 = 1$ 都可能输出 000，无法区分。

优先编码器 (Priority Encoder)

4 输入优先编码器设 D_3 优先级最高，输出 A_1, A_0 表示最高优先级的 1，V 表示是否至少存在一个 1。

行覆盖最小项数	D_3	D_2	D_1	D_0	A_1	A_0	V
1	0	0	0	0	X	X	0
1	0	0	0	1	0	0	1
2	0	0	1	X	0	1	1
4	0	1	X	X	1	0	1
8	1	X	X	X	1	1	1

从表可直接读出并优化：

$$A_1 = D_3 + D_2$$

$$A_0 = D_3 + \overline{D_2} D_1$$

$$V = D_3 + D_2 + D_1 + D_0$$

4.8 多路选择器 (Multiplexer)

定义

多路选择器执行数据选择：从一组信息输入中，根据选择线选择一路送到输出。典型 MUX 有：

- n 个选择输入 $S_{n-1}, \ldots, S_0$；
- 2^n 个信息输入 $I_{2^n-1}, \ldots, I_0$；
- 一个输出 Y。

也可设计 $m < 2^n$ 个信息输入、n 个选择输入的 MUX。

2-to-1 MUX

当 $S = 0$ 时选择 I_0，当 $S = 1$ 时选择 I_1：

$$Y = \overline{S} I_0 + S I_1$$

其结构可分为一个 1-to-2 译码器、两个使能电路和一个 2 输入 OR 门。推广到 2^n-to-1 MUX 时，可看作 n-to-2^n 译码器加 $2^n \times 2$ AND-OR 结构。

4-to-1 MUX 与扩展

4-to-1 MUX 的选择关系为：

S_0	S_1	F
0	0	A
0	1	B
1	0	C
1	1	D

结构上可使用 2-to-4 译码器和 4×2 AND-OR。64-to-1 MUX 对应 6-to-2^6 译码器和 $2^6 \times 2$ AND-OR。

位宽扩展 (Multiplexer Width Expansion)

MUX 不只能选择单个 bit，也可以选择 bit 向量。方法是把多个相同选择控制的 MUX 并联，例如 4-to-1 quad multiplexer 同时选择 4 位宽数据。

其他选择实现

- 使用三态逻辑替代 AND-OR，课件给出的门输入成本为 18。
- 把译码分布到三态驱动器中，门输入成本可降至 14；课件同时比较了 AND-OR 实现的 GN 为 22、AND-OR 替换为三态驱动器后的 GN 为 18。
- 使用传输门 (Transmission Gate) 实现 MUX，课件给出的门输入成本为 8，低于三态逻辑的 14 和门级逻辑的 18 或 22。

4.9 用 MUX 实现组合逻辑

若要实现 m 个 n 变量函数，可使用一个 m 位宽的 2^n-to-1 MUX：

1. 写出函数真值表。
2. 按真值表输入变量顺序，把函数输入接到 MUX 选择线 $S_{n-1}, \ldots, S_0$。
3. 将 MUX 输出标记为对应函数输出。
4. 按真值表把每个信息输入固定为 0 或 1；无关项可任选 0 或 1。

用 Dual 8-to-1 MUX 实现一位全加器

选择线为 X, Y, Z，两个输出分别为 S 和 C。按照全加器真值表设置两组信息输入：

X	Y	Z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

对应设置：

k	0	1	2	3	4	5	6	7
$I_{k,0}$ (S)	0	1	1	0	1	0	0	1
$I_{k,1}$ (C)	0	0	0	1	0	1	1	1

4.10 半加器与全加器补充 (Half Adder & Full Adder)

- **半加器 (Half Adder)**：输入 X, Y，输出和位 $S = X \oplus Y$，进位 $C = XY$。
- **全加器 (Full Adder)**：输入 X, Y, Z，其中 Z 可理解为低位进位。输出：

$$S = X \oplus Y \oplus Z$$

$$C = XY + XZ + YZ$$

- 全加器也可由两个半加器和一个 OR 门组成。

5 时序分析 (Timing Analysis)

5.1 电路延迟 (Circuit Delay)

理想情况下，逻辑门输出会在输入变化时立即变化；实际中，反相器、传输门等器件都有延迟和上升/下降时间。因此组合逻辑虽然没有状态，但输出不会瞬时稳定。

5.2 传播延迟与污染延迟

- **传播延迟 (Propagation Delay, T_{pd})**：输入变化到输出变化的最大延迟。
- **污染延迟 (Contamination Delay, T_{cd})**：输入变化到输出开始可能变化的最小延迟。

T_{pd} 和 T_{cd} 不同的原因包括：

- 上升沿和下降沿延迟不同；
- 多输入、多输出路径速度不同；
- 温度影响：电路热时变慢，冷时变快。

5.3 关键路径与最短路径

- **关键路径 (Critical/Longest Path)**：电路中传播延迟最大的路径。

$$T_{pd,\ circuit} = \sum_{\text{critical path}} T_{pd,\ element}$$

- **最短路径 (Shortest Path)**：电路中污染延迟最小的路径。

$$T_{cd,\ circuit} = \sum_{\text{shortest path}} T_{cd,\ element}$$

课件示例中，关键路径经过两个 AND 门和一个 OR 门：

$$T_{pd} = 2T_{pd_AND} + T_{pd_OR}$$

最短路径只经过最后一级 AND：

$$T_{cd} = T_{cd_AND}$$

5.4 竞争冒险与毛刺 (Race Hazard & Glitch)

- **Race Hazard**：同一输入变化经不同路径到达输出，路径延迟不同，可能导致短暂错误。
- **Glitch**：单个输入变化导致输出发生多次变化，即产生短暂脉冲。

课件示例可理解为 $F = A + \overline{A}$。理想逻辑恒为 1，但由于反相路径有延迟，当 A 从 1 变 0 时，直接路径先变 0，反相信号还没变 1，输出会短暂跌为 0，形成毛刺。

6 逻辑电路分析 (Analysis of Logic Circuits)

6.1 分析步骤

1. **写出电路布尔函数**：从输入信号开始，逐个门定义中间信号关系，并做必要优化。
2. **导出真值表**：明确输入和输出之间的关系。
3. **功能分析**：定义每个信号和整个电路的功能，必要时画时序图。
4. **验证**：检查最终分析或设计是否正确。

6.2 示例：分析一个组合逻辑电路

课件中电路的中间信号为：

$$P_1 = A \oplus B$$

$$P_2 = B \oplus C$$

$$P_3 = \overline{\overline{A} + \overline{B}} = AB$$

$$P_4 = \overline{A + C}$$

$$P_5 = \overline{P_1 P_2} = \overline{(A \oplus B)(B \oplus C)}$$

$$P_6 = P_3 + P_4 = AB + \overline{A + C}$$

最终输出为：

$$F = \overline{P_5 P_6} = \overline{\overline{(A \oplus B)(B \oplus C)} \cdot (AB + \overline{A + C})}$$

布尔函数化简

由德摩根律：

$$F = (A \oplus B)(B \oplus C) + \overline{AB + \overline{A + C}}$$

继续展开并化简：

$$\begin{aligned}
F &= (A\overline{B} + \overline{A}B)(B\overline{C} + \overline{B}C) + (\overline{A} + \overline{B})(A + C) \\
&= A\overline{B}C + \overline{A}B\overline{C} + \overline{A}C + A\overline{B} + \overline{B}C \\
&= \overline{A}B\overline{C} + \overline{A}C + A\overline{B} + \overline{B}C \\
&= \overline{A}C + A\overline{B} + \overline{A}B\overline{C} \\
&= \overline{A}(C + B\overline{C}) + A\overline{B} \\
&= \overline{A}C + \overline{A}B + A\overline{B} \\
&= \overline{A}C + (A \oplus B)
\end{aligned}$$

A	B	C	$A \oplus B$	$\overline{A}C$	F
0	0	0	0	0	0
0	0	1	0	1	1
0	1	0	1	0	1
0	1	1	1	1	1
1	0	0	1	0	1
1	0	1	1	0	1
1	1	0	0	0	0
1	1	1	0	0	0

从真值表可读出：

$$F = 1 \Longleftrightarrow A \neq B \text{ 或 } A < C$$

因此原电路不是最优实现。更简洁的实现是一个 XOR 门产生 $A \oplus B$，一个反相器和 AND 门产生 $\overline{A}C$，最后用 OR 门合并。

7 复习小结 (Summary)

- Verilog HDL 用模块、端口、数据类型和赋值/过程语句描述硬件；要区分可综合逻辑和仅用于仿真的 testbench 代码。
- 组合逻辑是无记忆电路，输出只依赖当前输入；设计流程包括规格、公式化、优化、技术映射和验证。
- 常用组合功能块包括基本逻辑功能、使能、译码器、编码器、优先编码器、多路选择器、解复用器、七段显示译码器、半加器和全加器。
- 译码器适合从最小项实现函数；MUX 适合直接按真值表固定信息输入实现函数。

- 实际电路存在传播延迟和污染延迟，关键路径决定最大延迟，最短路径影响污染延迟；路径延迟差会导致竞争冒险和毛刺。
- 分析已有电路时，应先逐级列中间信号，再化简、列真值表、解释功能并验证是否可优化。